МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ

Федеральное государственное автономное

образовательное учреждение

высшего образования

«Национальный исследовательский университет ИТМО»

(ФГАОУ ВО «ИТМО»)

Факультет: «Факультет программной инженерии и компьютерной техники»

Дисциплина: «Программирование на Python».

Лабораторная работа №2

Тема: «Численные вычисления и анализ данных с использованием NumPy».

Группа P3121

Выполнила:

Шукалович Екатерина Андреевна

Проверил:

Жуков Николай Николаевич

Санкт-Петербург 2026

Задача: освоить базовые операции численных вычислений и анализа данных с использованием библиотеки NumPy.

Решение:

1. Настройка и активация виртуального окружения:

   python -m venv numpy_env

   source numpy_env/bin/activate

   pip install numpy matplotlib seaborn pandas pytest

2. Векторные операции (на примере одного вектора):

```
def create_vector() -> np.ndarray:
    """
    Создать одномерный массив целых чисел от 0 до 9 включительно.

    Returns:
        np.ndarray: Массив shape=(10,) с элементами [0, 1, ..., 9].
    """
    return np.arange(10)
```

3. Матричные операции (на примере определителя квадратной матрицы):

```
def matrix_determinant(a: np.ndarray) -> float:
    """
    Вычислить определитель квадратной матрицы.
    Args:
        a (np.ndarray): Квадратная матрица.
    Returns:
        float: Значение определителя.
    Raises:
        ValueError: Если матрица не квадратная.
    """
    if a.ndim != 2 or a.shape[0] != a.shape[1]:
        raise ValueError(f"Expected square matrix, got shape {a.shape}")
    return float(np.linalg.det(a))
```

4. Статистический анализ (с выгрузкой файла StudentsPerformance.csv.xls):

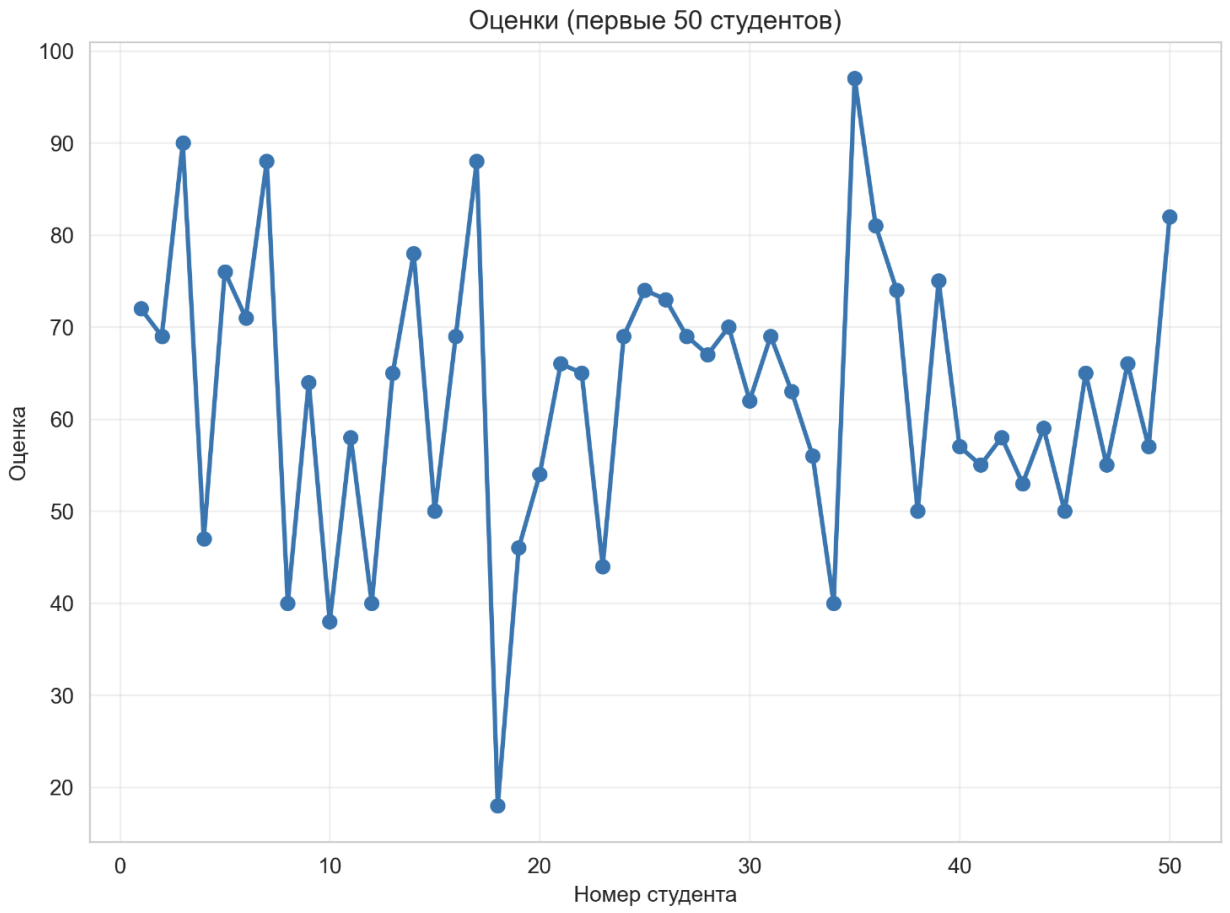
```
def load_dataset(path: str = "data/StudentsPerformance.csv.xls") -> np.ndarray:
    """
    Загрузить данные из CSV-файла и вернуть их в виде NumPy-массива.
    Автоматически выбирает только числовые колонки.
    Args:
        path (str): Путь к файлу CSV.
    Returns:
        np.ndarray: Двумерный массив с числовыми данными.
    Raises:
```

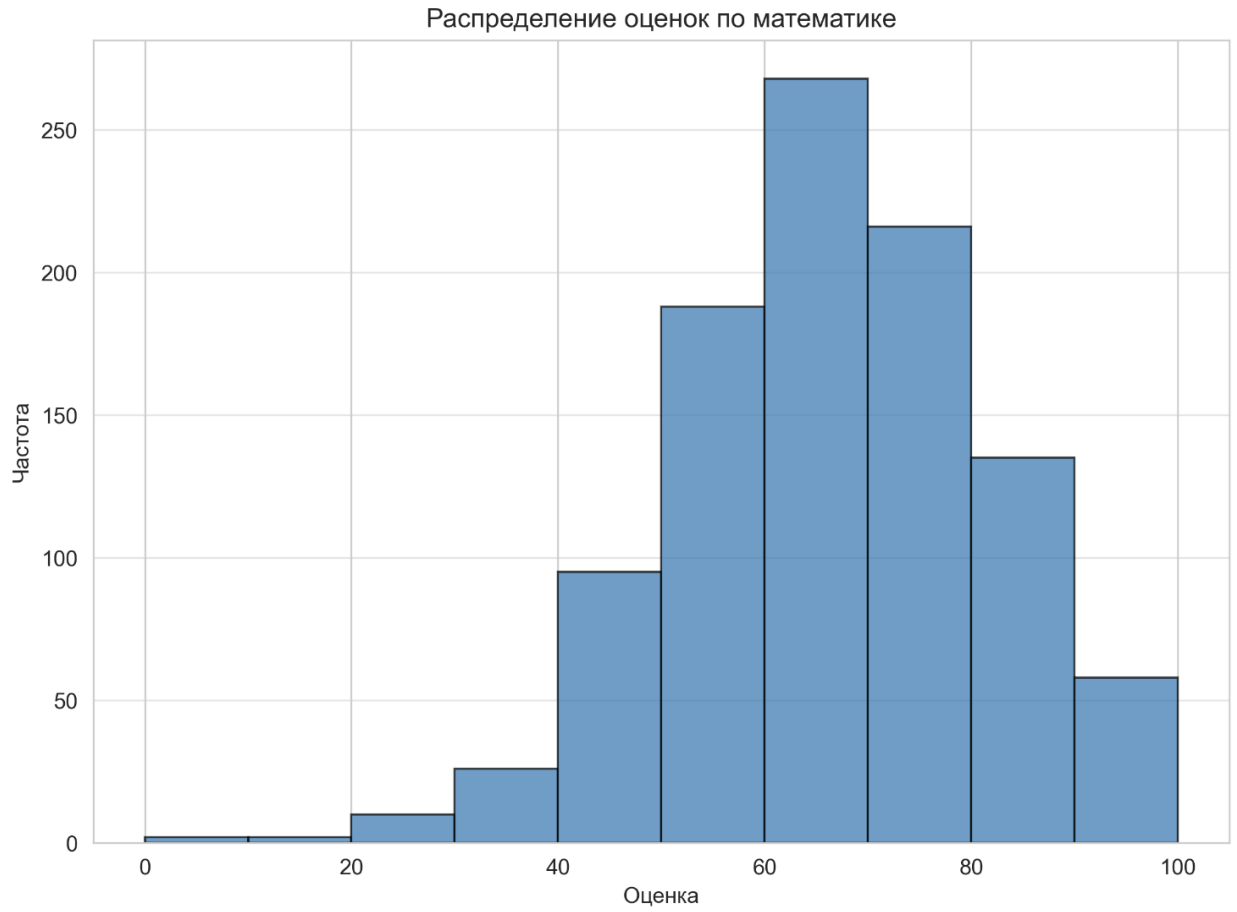
```
        FileNotFoundError: Если файл не найден.
        ValueError: Если в файле нет числовых колонок.
    """
    if not os.path.exists(path):
        raise FileNotFoundError(f"Dataset file not found: {path}")

    df = pd.read_csv(path)

    # Оставляем только числовые колонки
    numeric_df = df.select_dtypes(include=[np.number])

    if numeric_df.empty:
        raise ValueError("No numeric columns found in the dataset")

    return numeric_df.to_numpy()
```

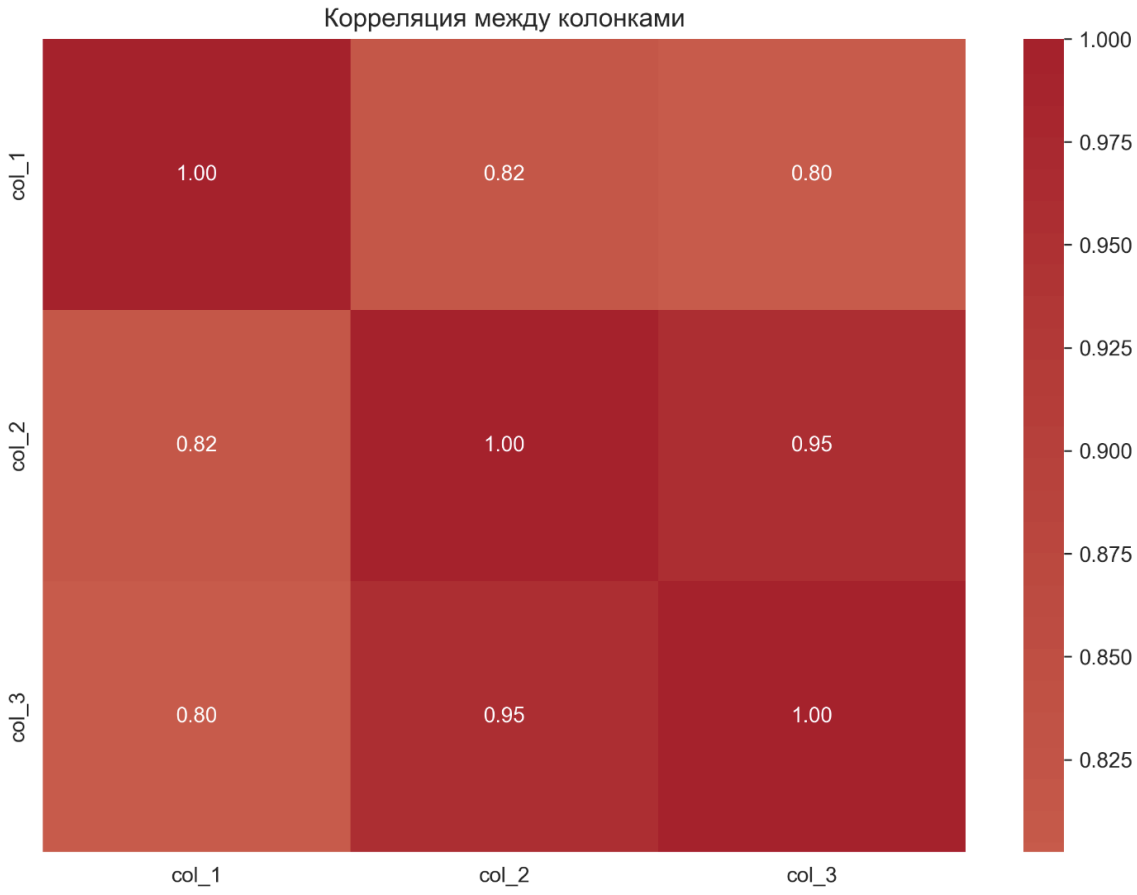
5. Визаулизация данных из файла производится с помощью графиков (на примере графика зависимости):

```
def plot_line(
    x: np.ndarray,
    y: np.ndarray,
    output_path: str = "plots/line_plot.png",
    title: str = "Зависимость: студент → оценка по математике",
    xlabel: str = "Номер студента",
    ylabel: str = "Оценка",
) -> None:
    """
    Построить и сохранить линейный график зависимости y от x.
    Args:
        x (np.ndarray): Массив значений по оси X.
        y (np.ndarray): Массив значений по оси Y.
        output_path (str): Путь для сохранения изображения.
        title (str): Заголовок графика.
        xlabel (str): Подпись оси X.
        ylabel (str): Подпись оси Y.
    Raises:
        ValueError: Если массивы x и y имеют разную длину.
    """
    if x.shape != y.shape:
        raise ValueError(f"Shapes mismatch: {x.shape} vs {y.shape}")

    os.makedirs(os.path.dirname(output_path), exist_ok=True)

    plt.figure()
    plt.plot(x, y, marker="o", linewidth=2, markersize=6)
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.savefig(output_path, dpi=300)
plt.close()
```


Оценки (первые 50 студентов)
Оценка
Номер студента
100
90
80
70
60
50
40
30
20
0
10
20
30
40
50


Распределение оценок по математике
Частота
Оценка
250
200
150
100
50
0
0
20
40
60
80
100



6. Тестирование с помощью pytest (на примере выполнения свойств матрицы):

```
def test_matrix_inverse() -> None:
"""Проверка: A × A⁻¹ = I (единичная матрица)."""
A = np.array([[1, 2], [3, 4]])
invA = matrix_inverse(A)
# Используем allclose для учёта погрешностей плавающей точки
assert np.allclose(A @ invA, np.eye(2), atol=1e-10)
```

Нюансы при решении:

1) **Типы данных из csv-файла**
   При загрузке датасета (с текстовыми колонками) pd.read_csv().to_numpy() возвращает массив с dtype=object, что ломает математические операции.
2) **Нормализация при постоянных данных**
   Если min == max, то по формуле получается деление на 0